

1 . To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Aim : To implement double quotes in a shell parser so that:

- Spaces inside double quotes are preserved.
- Variables such as `$USER` are expanded.
- Quoted text is treated as one argument.
- Nested tokens can be parsed correctly.
- Unmatched double quotes are detected.
- Quoted commands can be tested.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_ARGS 20
#define MAX_LEN 200

/*
 * Expand environment variables such as $USER
 */
void expand_variable(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            while ((input[i] >= 'A' && input[i] <= 'Z') ||
                   (input[i] >= 'a' && input[i] <= 'z') ||
                   (input[i] >= '0' && input[i] <= '9') ||
                   input[i] == '_')
            {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL)
            {
                for (int x = 0; value[x] != '\0'; x++)
                {
                    if (j < MAX_LEN - 1)
                        output[j++] = value[x];
                }
            }
        }
        else
            output[j++] = input[i++];
    }
}
```

```
        else
        {
            if (j < MAX_LEN - 1)
                output[j++] = input[i];

            i++;
        }
    }
    output[j] = '\0';
}

int parse_command(char *input, char args[MAX_ARGS][MAX_LEN])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        /* Skip spaces */
        while (input[i] == ' ')
            i++;

        if (input[i] == '\0')
            break;

        int j = 0;

        /*
         * Double quoted string
         */
        if (input[i] == '"')
        {
            char quoted[MAX_LEN];

            i++;

            while (input[i] != '"' && input[i] != '\0')
            {
                if (j < MAX_LEN - 1)
                    quoted[j++] = input[i];

                i++;
            }

            /*
             * Check for unmatched double quote
            */
        }
    }
}
```

```

navaneeth@NAVANEETH:~/OSSP$ cd ~/OSSP
navaneeth@NAVANEETH:~/OSSP$ nano task2_single_quotes.c
navaneeth@NAVANEETH:~/OSSP$ gcc -Wall -Wextra task2_single_quotes.c -o task2_single_quotes
navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo Hello

Parsed arguments:
Argument 1: echo
Argument 2: Hello
navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo 'Hello World'

Parsed arguments:
Argument 1: echo
Argument 2: Hello World
navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo '$USER'

Parsed arguments:
Argument 1: echo
Argument 2: $USER
navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo 'Hello Linux World'

Parsed arguments:
Argument 1: echo
Argument 2: Hello Linux World
navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo 'Hello $USER * ?'

Parsed arguments:
Argument 1: echo
Argument 2: Hello $USER * ?
navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo ''

Parsed arguments:
Argument 1: echo
Argument 2:

navaneeth@NAVANEETH:~/OSSP$ ./task2_single_quotes
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.

navaneeth@NAVANEETH:~/OSSP$ █

```

Output :

2 . To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Aim :

To implement double quotes in a shell parser so that:

- Spaces inside double quotes are preserved.
- Variables such as `$USER` are expanded.
- Quoted text is treated as one argument.
- Nested tokens can be parsed correctly.
- Unmatched double quotes are detected.
- Quoted commands can be tested.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_ARGS 20
#define MAX_LEN 200

/*
 * Expand environment variables such as $USER
 */
void expand_variable(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            while ((input[i] >= 'A' && input[i] <= 'Z') ||
                   (input[i] >= 'a' && input[i] <= 'z') ||
                   (input[i] >= '0' && input[i] <= '9') ||
                   input[i] == '_')
            {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL)
            {

```

```
        if (value != NULL)
        {
            for (int x = 0; value[x] != '\0'; x++)
            {
                if (j < MAX_LEN - 1)
                    output[j++] = value[x];
            }
        }
    }
    else
    {
        if (j < MAX_LEN - 1)
            output[j++] = input[i];

        i++;
    }
}

output[j] = '\0';
}

int parse_command(char *input, char args[MAX_ARGS][MAX_LEN])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        /* Skip spaces */
        while (input[i] == ' ')
            i++;

        if (input[i] == '\0')
            break;

        int j = 0;

        /*
```

GNU nano 7.2

```
    }

    args[argc][j] = '\\0';
    argc++;
}

if (argc >= MAX_ARGS)
    break;
}

return argc;

int main()
{
    char input[500];
    char args[MAX_ARGS][MAX_LEN];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\\n")] = '\\0';
    int argc = parse_command(input, args);

    if (argc == -1)
        return 1;

    printf("\\nParsed arguments:\\n");

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\\n", i + 1, args[i]);
    }

    return 0;
}
```

```
navaneeth@NAVANEETH:~/OSSP$ cd ~/OSSP
navaneeth@NAVANEETH:~/OSSP$ nano task3_double_quotes.c
navaneeth@NAVANEETH:~/OSSP$ gcc -Wall -Wextra task3_double_quotes.c -o task3_double_quotes
navaneeth@NAVANEETH:~/OSSP$ ./task3_double_quotes
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

Child Process
PID = 1080

Hello

Child process completed.

myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 1081

Hello World

Child process completed.

myShell> echo "$USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: $USER

Child Process
PID = 1082

Hello $USER

Child process completed.

myShell> echo navaneeth

--- Parsed Arguments ---
Argument 1: echo
Argument 2: navaneeth

Child Process
PID = 1082

Hello navaneeth

Child process completed.
```

Output :

```
myShell> echo "Home directory is $HOME"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Home directory is /home/alekhya

Child Process
PID = 1083

Home directory is /home/alekhya

Child process completed.

myShell> echo "Hello $USER Welcome"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello alekhya Welcome

Child Process
PID = 1084

Hello alekhya Welcome

Child process completed.

myShell> echo "Hello World" "Linux System"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Argument 3: Linux System

Child Process
PID = 1085

Hello World Linux System

Child process completed.

myShell> echo "User: $USER, Home: $HOME"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: User: alekhya, Home: /home/alekhya
```

```
myShell> echo "User: $USER, Home: $HOME"
```

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: User: alekhya, Home: /home/alekhya
```

```
Child Process
```

```
PID = 1086
```

```
User: alekhya, Home: /home/alekhya
```

```
Child process completed.
```

```
myShell> echo "Hello World
```

```
Syntax Error: Unmatched double quote.
```

```
myShell> echo "
```

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2:
```

```
Child Process
```

```
PID = 1089
```

```
Child process completed.
```

```
myShell> echo "Hello Linux"
```

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: Hello Linux
```

```
Child Process
```

```
PID = 1090
```

```
Hello Linux
```

```
Child process completed.
```